

Mustafa Abbas Abokhadher

2240002889

7MA2

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

import matplotlib.pyplot as plt
import matplotlib
import numpy as np
import skimage as ski

matplotlib.rcParams['font.size'] = 18

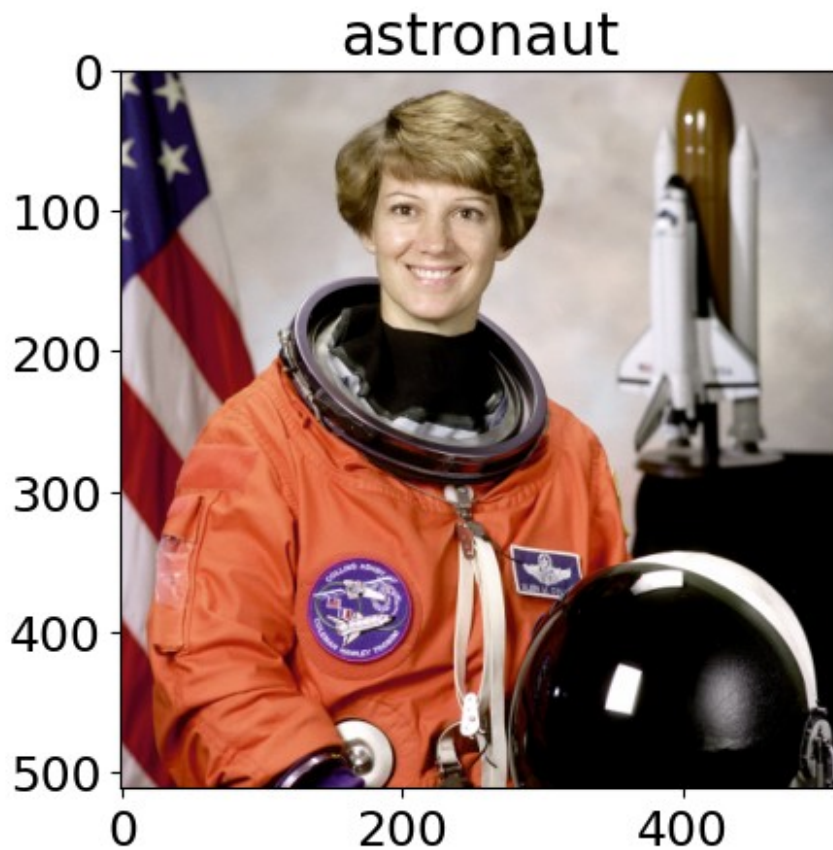
images = (
    'astronaut',
    'binary_blobs',
    'brick',
    'colorwheel',
    'camera',
    'cat',
    'checkerboard',
    'clock',
    'coffee',
    'coins',
    'eagle',
    'grass',
    'gravel',
    'horse',
    'logo',
    'page',
    'text',
    'rocket',
)

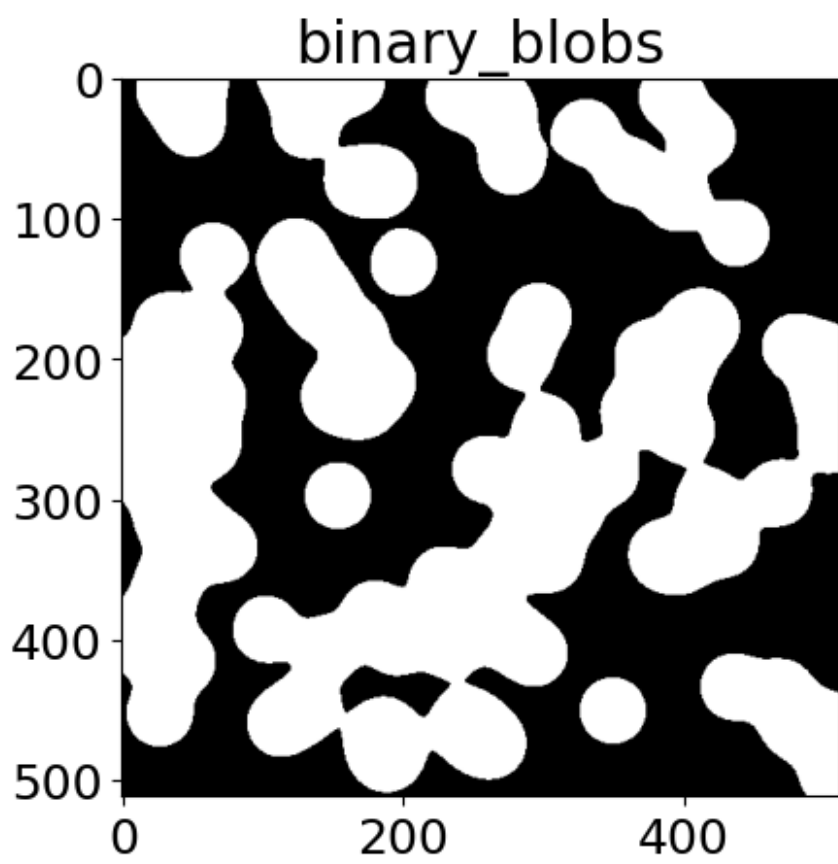
for name in images:
    caller = getattr(ski.data, name)
    image = caller()
    plt.figure()
    plt.title(name)
```

```
if image.ndim == 2:  
    plt.imshow(image, cmap=plt.cm.gray)  
else:  
    plt.imshow(image)
```

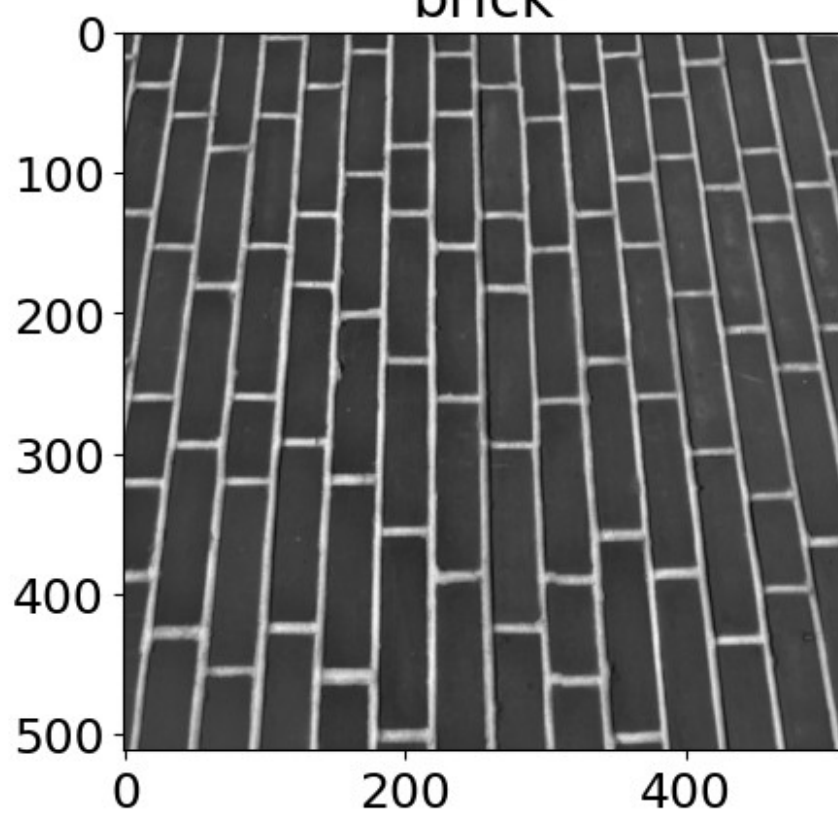
```
plt.show()
```

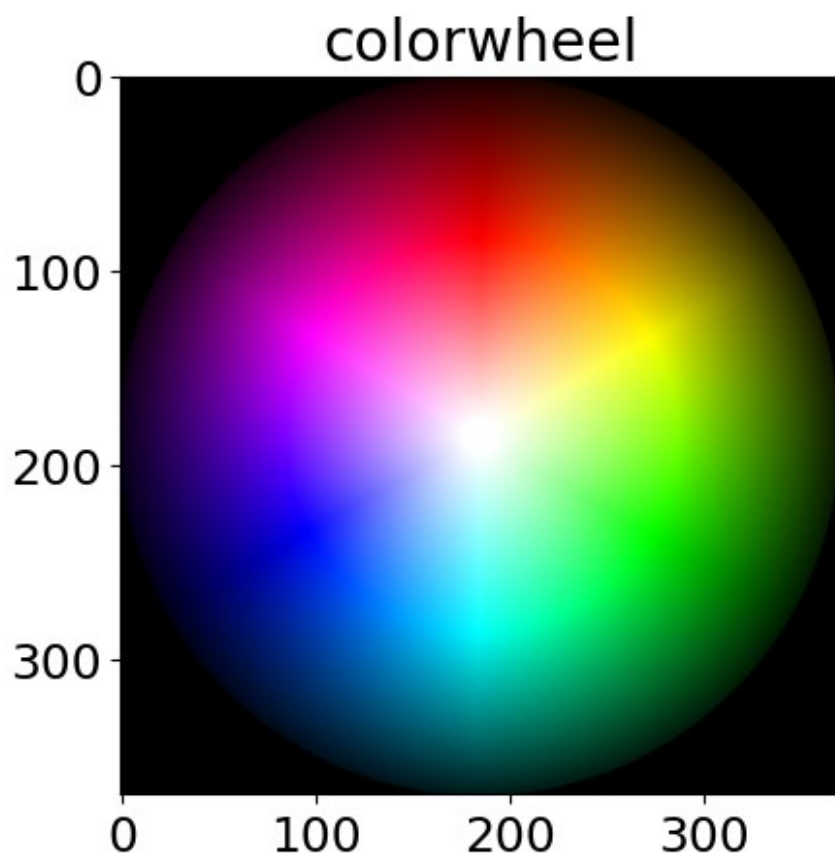
```
c:\Users\musta\AppData\Local\Programs\Python\Python314\Lib\site-  
packages\tqdm\auto.py:21: TqdmWarning: IProgress not found. Please  
update jupyter and ipywidgets. See  
https://ipywidgets.readthedocs.io/en/stable/user\_install.html  
from .autonotebook import tqdm as notebook_tqdm
```



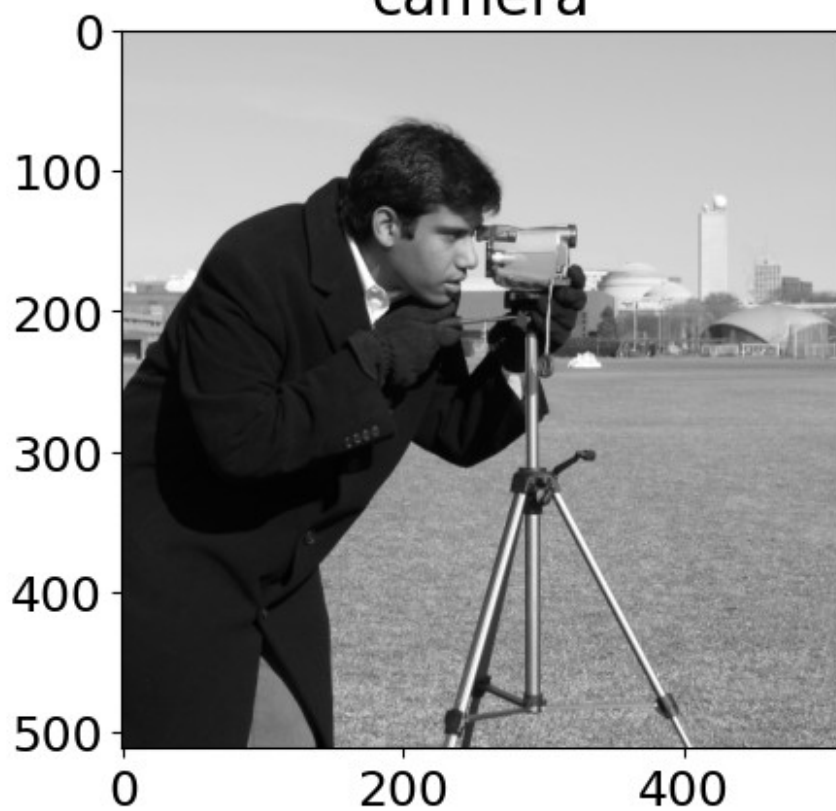


brick

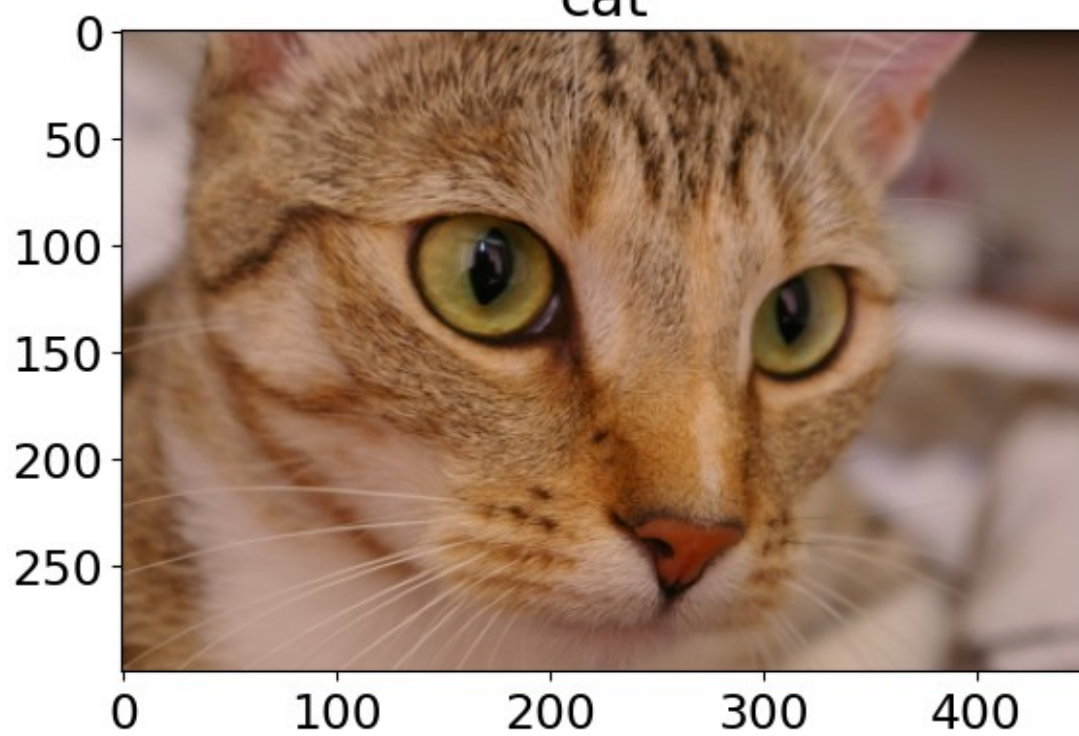


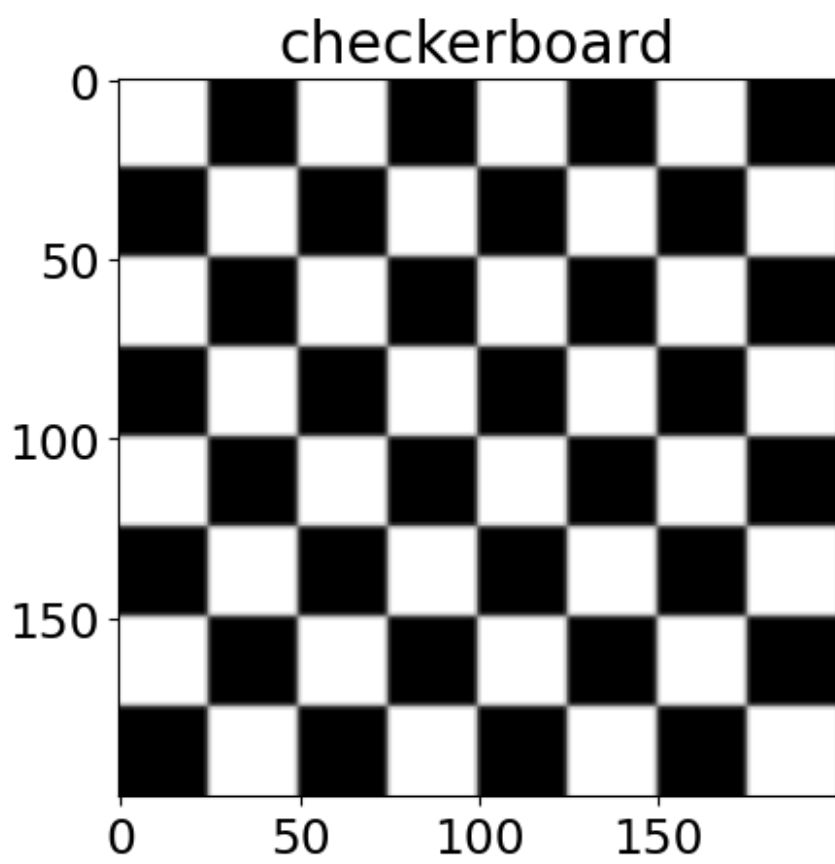


camera

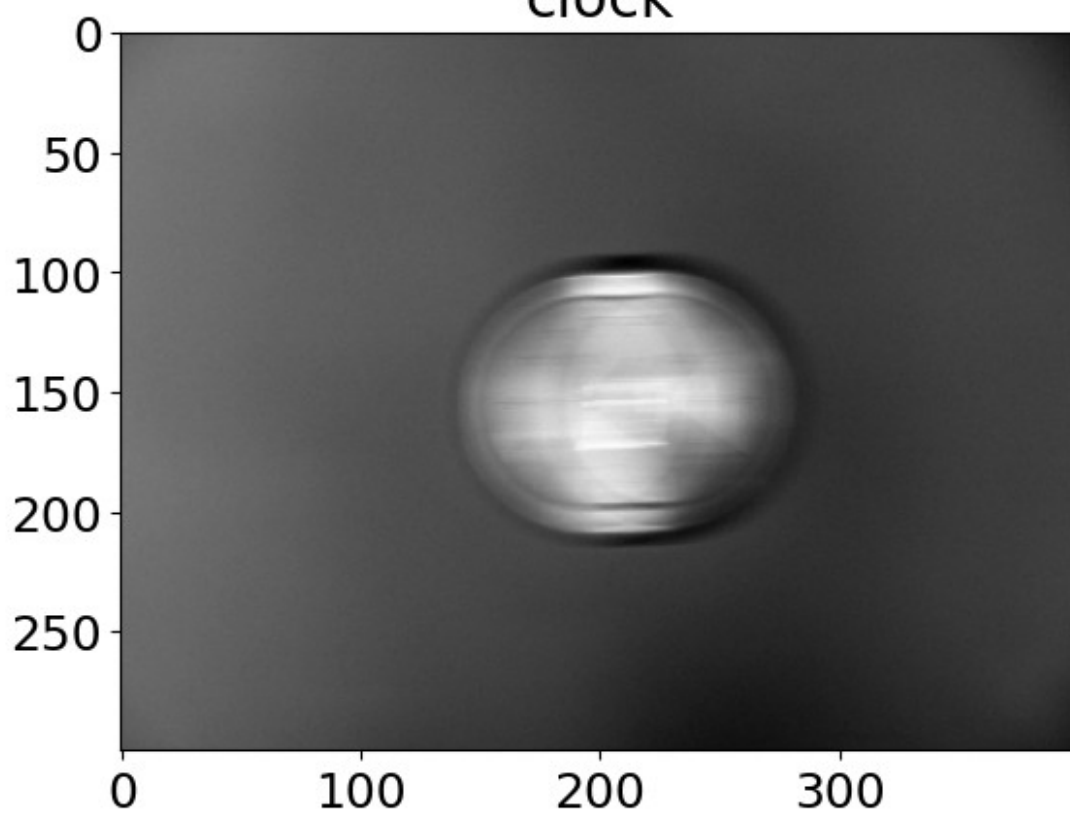


cat

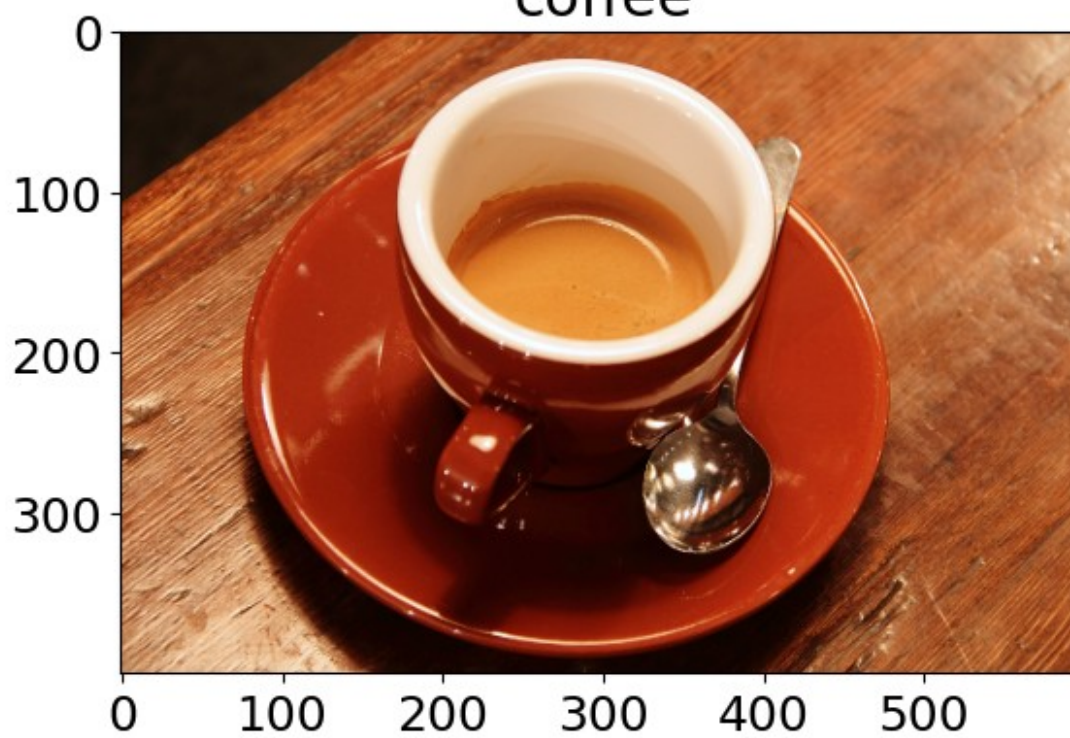




clock

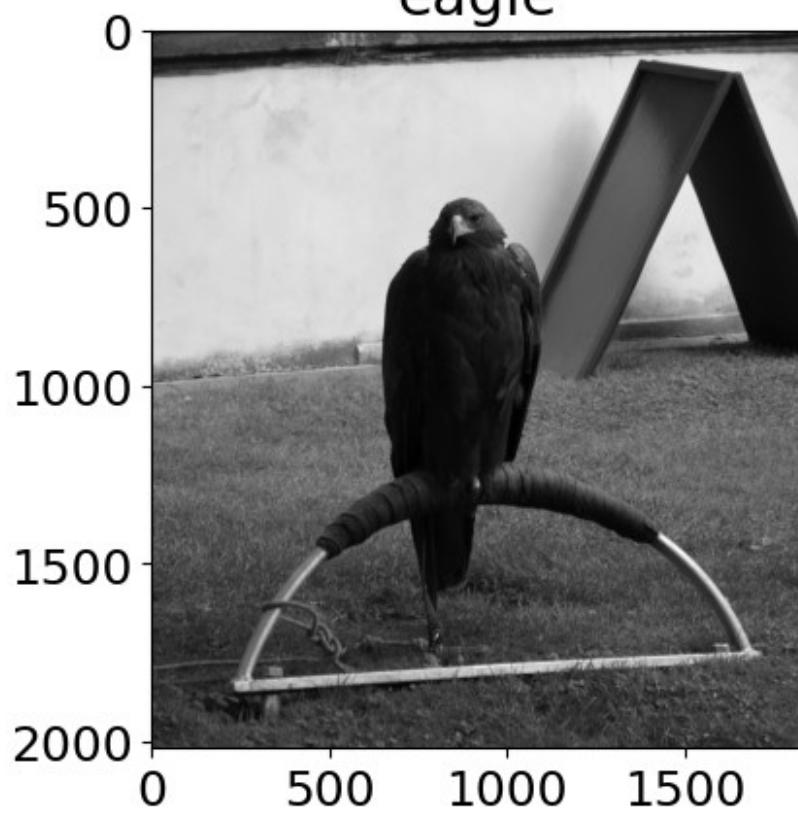


coffee

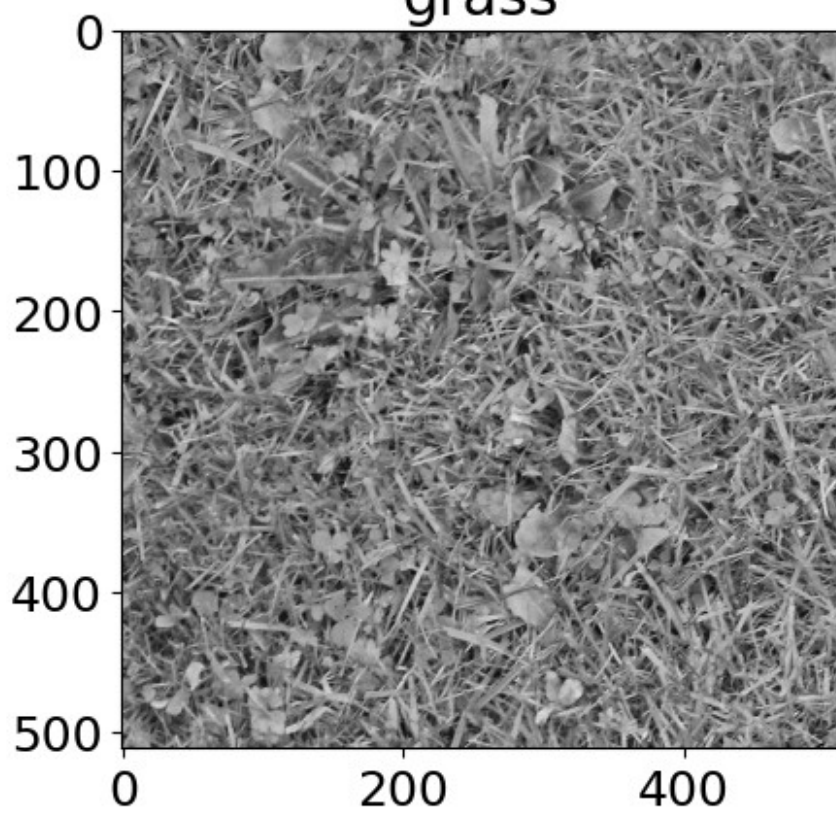


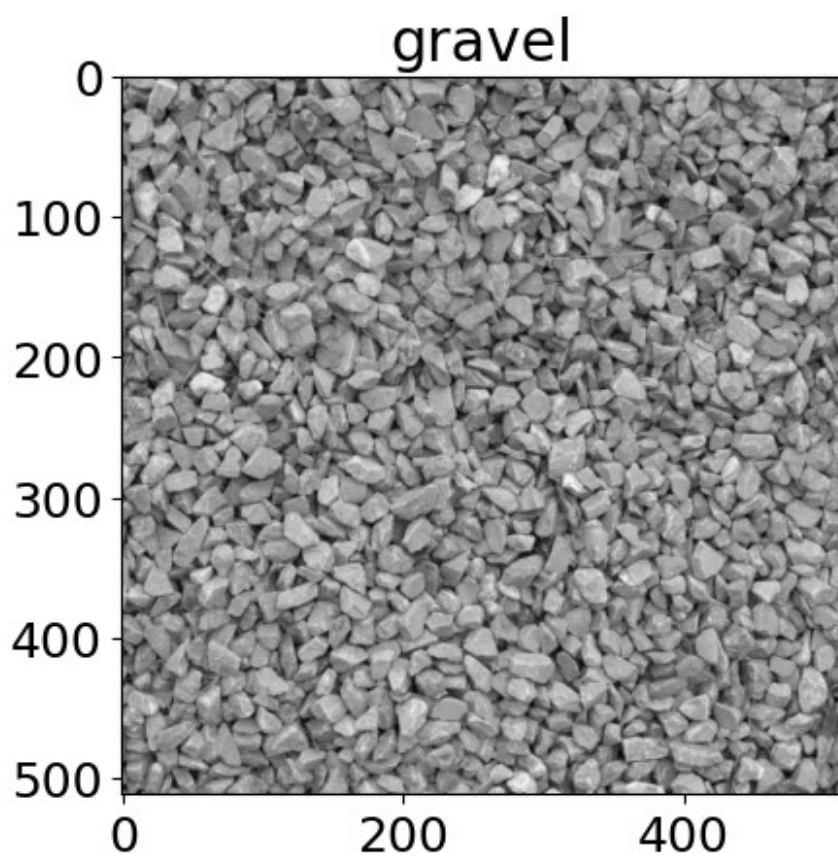
A 4x6 grid of 24 ancient coins, likely Roman, displayed on a dark background. The coins are arranged in four rows and six columns. The vertical axis is labeled 'COINS' and ranges from 0 to 300. The horizontal axis is labeled '0', '100', '200', and '300'.

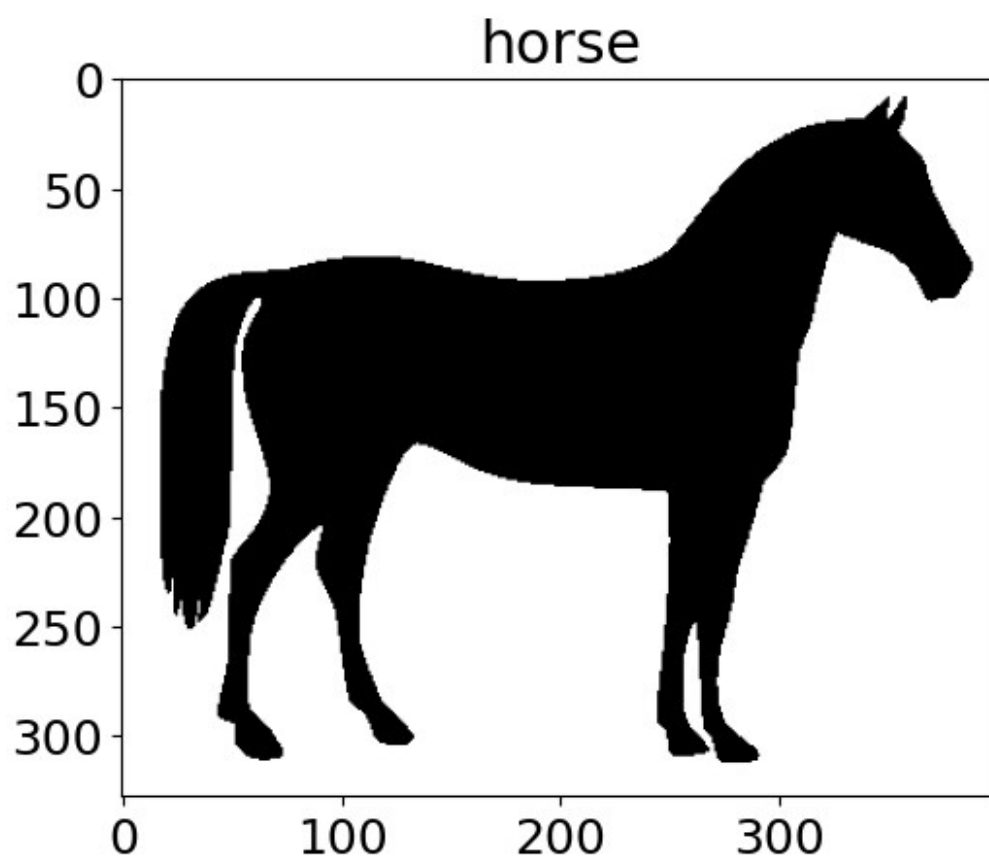
eagle

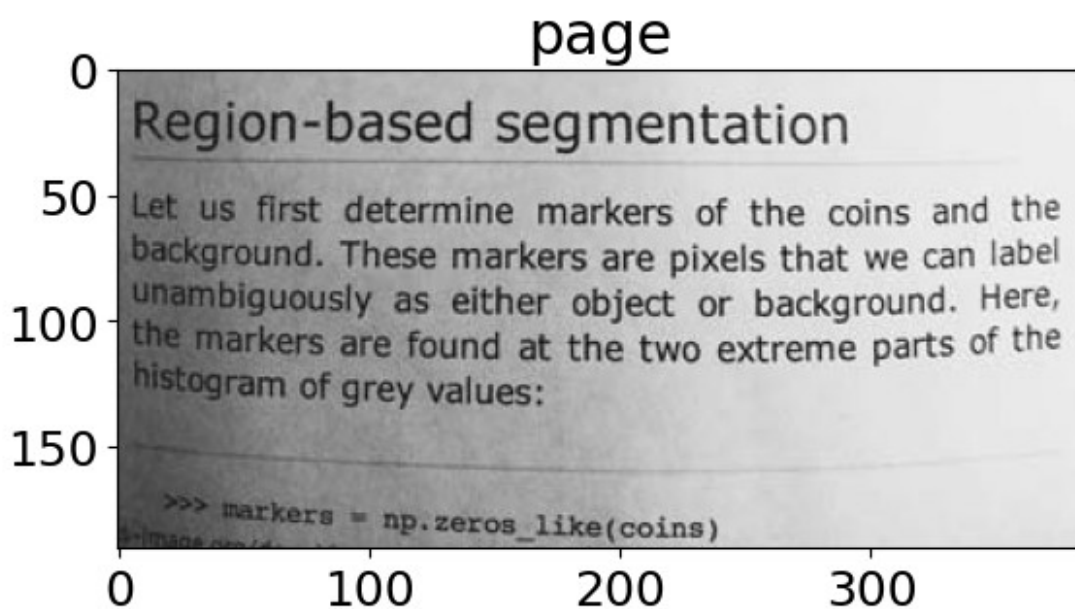
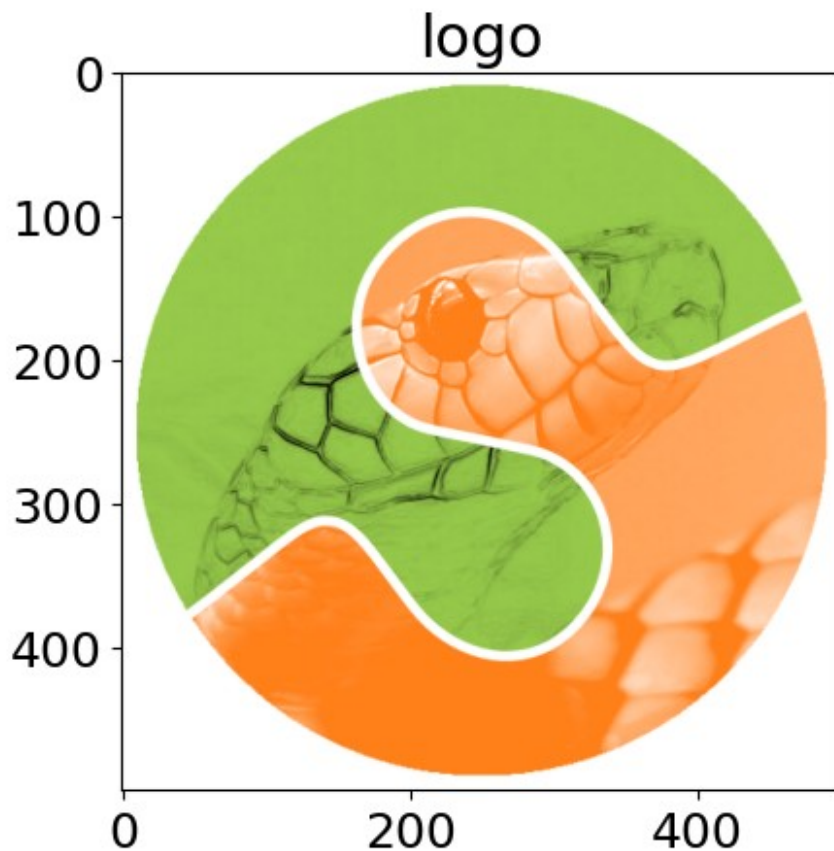


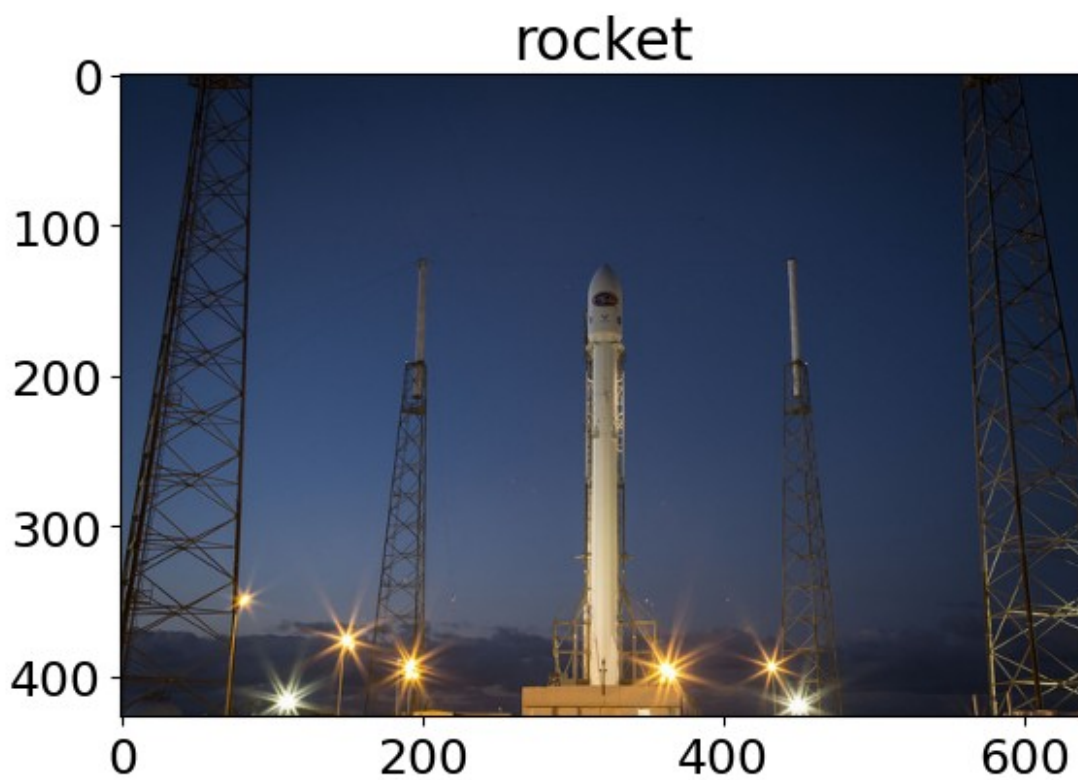
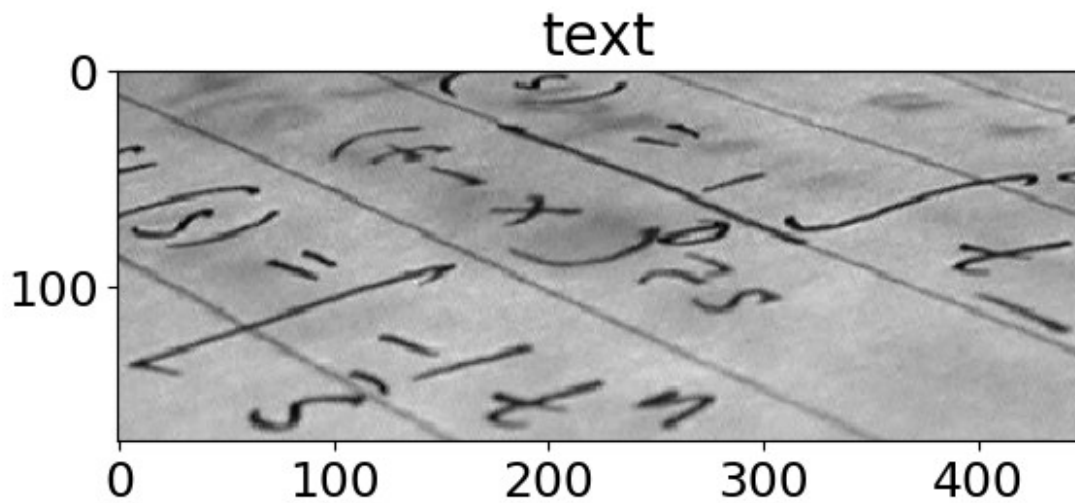
grass











1. Image Sampling and Quantization

```
def sample_image(image, factor):  
    """  
    Downsamples the image by the given factor.  
    Args:  
    image (numpy array): Original image.  
    factor (int): Factor by which to downsample.  
    Returns:
```

```

    numpy array: Downsampled image.
    """
    height, width = image.shape[:2]
    sampled_image = cv2.resize(image, (width // factor, height // factor),
                               interpolation=cv2.INTER_NEAREST)
    return sampled_image

def quantize_image(image, levels):
    """
    Reduces the number of grayscale levels in the image.
    Args:
    image (numpy array): Original image.
    levels (int): Number of grayscale levels.
    Returns:
    numpy array: Quantized image.
    """

    quantized_image = np.floor(image / (256 // levels)) * (256 // levels)
    quantized_image = quantized_image.astype(np.uint8)
    return quantized_image

def plot_images(original, sampled, quantized):
    """
    Plots the original, sampled, and quantized images side by side.
    Args:
    original (numpy array): Original image.
    sampled (numpy array): Sampled image.
    quantized (numpy array): Quantized image.
    """

    plt.figure(figsize=(12, 4))
    plt.subplot(1, 3, 1)
    plt.imshow(original, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')
    plt.subplot(1, 3, 2)
    plt.imshow(sampled, cmap='gray')
    plt.title('Sampled Image')
    plt.axis('off')
    plt.subplot(1, 3, 3)
    plt.imshow(quantized, cmap='gray')
    plt.title('Quantized Image')
    plt.axis('off')
    plt.show()

image_path='images/lena_gray_256.tif'
sampling_factor=14
quantization_levels=9

# Load image in grayscale

```

```

original_image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
if original_image is None:
    print(f"Error: Unable to load image at {image_path}")
# Sample and quantize
sampled_image = sample_image(original_image, sampling_factor)
quantized_image = quantize_image(original_image,
quantization_levels)
# Plot results
plot_images(original_image, sampled_image, quantized_image)

```

Original Image



Sampled Image



Quantized Image



2. Arithmetic Operations

```

import numpy as np
from PIL import Image

img1 = Image.open('images/lena_gray_256.tif').convert('L')
img2 = Image.open('images/camera.tif').convert('L')

resize = (400, 400)
img1 = img1.resize(resize, Image.Resampling.LANCZOS)
img2 = img2.resize(resize, Image.Resampling.LANCZOS)

im1arr = np.asarray(img1, dtype=np.int16)
im2arr = np.asarray(img2, dtype=np.int16)

addition = np.clip(im1arr + im2arr, 0, 255).astype(np.uint8)

resultImage = Image.fromarray(addition)
resultImage.show()

```

3. Sets and Logical Operations

```

img3 = Image.open('images/A.jpeg').convert('L')
img3.show()

img4 = Image.open('images/B.jpeg').convert('L')

```

```

img4.show()

resize = (400, 400)
img3 = img3.resize(resize, Image.Resampling.LANCZOS)
img4 = img4.resize(resize, Image.Resampling.LANCZOS)

im3arr = np.asarray(img3)
im4arr = np.asarray(img4)

union = im4arr | im3arr

resultImage2 = Image.fromarray(union)
resultImage2.show()

```

Task#1

```

def sample_image(image, factor):
    height, width = image.shape[:2]
    sampled_image = cv2.resize(image, (width // factor, height //
                                     factor),
                                interpolation=cv2.INTER_NEAREST)
    return sampled_image

def quantize_image(image, levels):
    quantized_image = np.floor(image / (256 // levels)) * (256 //
                                                             levels)
    quantized_image = quantized_image.astype(np.uint8)
    return quantized_image

def plot_images(original, sampled, quantized, sampling_factor,
                 quantization_levels):
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 3, 1)
    plt.imshow(original, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')

    plt.subplot(1, 3, 2)
    plt.imshow(sampled, cmap='gray')
    plt.title(f'Sampled Image (factor={sampling_factor})')
    plt.axis('off')

    plt.subplot(1, 3, 3)
    plt.imshow(quantized, cmap='gray')
    plt.title(f'Quantized Image (levels={quantization_levels})')
    plt.axis('off')

```

```

plt.show()

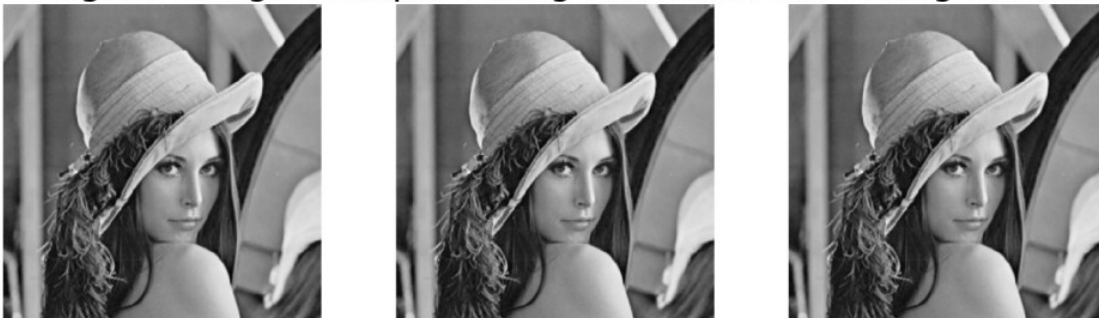
image_path = 'images/lena_gray_256.tif'
original_image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

if original_image is None:
    print(f"Error: Unable to load image at {image_path}")
else:
    test_params = [
        (2, 256),
        (4, 32),
        (8, 8),
        (16, 2), ]

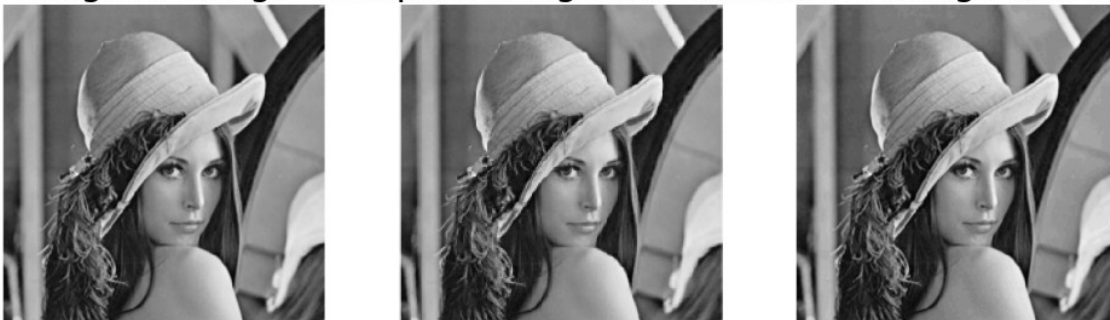
    for sampling_factor, quantization_levels in test_params:
        sampled_image = sample_image(original_image, sampling_factor)
        quantized_image = quantize_image(original_image,
        quantization_levels)
        plot_images(original_image, sampled_image, quantized_image,
        sampling_factor, quantization_levels)

```

Original Image Sampled Image (factor=2) Quantized Image (levels=256)



Original Image Sampled Image (factor=4) Quantized Image (levels=32)



Original Image Sampled Image (factor=8) Quantized Image (levels=8)



Original Image Sampled Image (factor=16) Quantized Image (levels=2)



Task#2

```
img1 = Image.open('images/lena_gray_256.tif').convert('L')
img2 = Image.open('images/camera.tif').convert('L')

resize = (400, 400)
img1 = img1.resize(resize, Image.Resampling.LANCZOS)
img2 = img2.resize(resize, Image.Resampling.LANCZOS)

im1arr = np.asarray(img1).astype(np.int16)
im2arr = np.asarray(img2).astype(np.int16)

subtraction = im1arr - im2arr
subtraction = np.clip(subtraction, 0, 255).astype(np.uint8)
subResultImage = Image.fromarray(subtraction)
subResultImage.show(title="Subtraction Result")

addConstant = im1arr + 175
addConstant = np.clip(addConstant, 0, 255).astype(np.uint8)
addConstResultImage = Image.fromarray(addConstant)
addConstResultImage.show(title="Addition with Constant (175)")

im1arr_u8 = np.asarray(img1)
im2arr_u8 = np.asarray(img2)
```

```
set_difference = cv2.bitwise_and(im1arr_u8,  
cv2.bitwise_not(im2arr_u8))  
setDiffResultImage = Image.fromarray(set_difference)  
setDiffResultImage.show(title="Set Difference (A - B)")  
  
symmetric_difference = cv2.bitwise_xor(im1arr_u8, im2arr_u8)  
symDiffResultImage = Image.fromarray(symmetric_difference)  
symDiffResultImage.show(title="Symmetric Difference (A XOR B)")  
  
intersection = cv2.bitwise_and(im1arr_u8, im2arr_u8)  
intersectResultImage = Image.fromarray(intersection)  
intersectResultImage.show(title="Intersection (A AND B)")
```